

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Load the Titanic dataset

```
In [ ]: # Load the Titanic dataset
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"

df = pd.read_csv(url)

# Display first 5 rows
df.head()
```

```
Out[ ]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adul
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	

```
In [5]: df.shape
```

```
Out[5]: (891, 15)
```

```
In [6]: df.columns
```

```
Out[6]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
               'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
               'alive', 'alone'],
              dtype='str')
```

```
In [7]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    str
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    str
8   class       891 non-null    str
9   who         891 non-null    str
10  adult_male  891 non-null    bool
11  deck        203 non-null    str
12  embark_town 889 non-null    str
13  alive        891 non-null    str
14  alone       891 non-null    bool
dtypes: bool(2), float64(2), int64(4), str(7)
memory usage: 92.4 KB

```

```
In [8]: df.describe()
```

```
Out[8]:
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Part I: Frequency Analysis

Absolute Frequency

```
In [9]: df["class"].value_counts()
```

```
Out[9]: class
Third    491
First    216
Second   184
Name: count, dtype: int64
```

Relative Frequency

```
In [10]: df["class"].value_counts(normalize=True)
```

```
Out[10]: class
Third      0.551066
First      0.242424
Second     0.206510
Name: proportion, dtype: float64
```

Relative Frequency in Percentage

```
In [11]: df["class"].value_counts(normalize=True) * 100
```

```
Out[11]: class
Third      55.106622
First      24.242424
Second     20.650954
Name: proportion, dtype: float64
```

Cumulative Frequency

```
In [12]: df["class"].value_counts().cumsum()
```

```
Out[12]: class
Third      491
First      707
Second     891
Name: count, dtype: int64
```

Create a Proper Frequency Table

```
In [13]: absolute = df["class"].value_counts().sort_index()

relative = (
    df["class"]
    .value_counts(normalize=True)
    .sort_index()
    * 100
).round(2)

cumulative = absolute.cumsum()

frequency_table = pd.DataFrame({
    "Absolute Frequency": absolute,
    "Relative Frequency (%)": relative,
    "Cumulative Frequency": cumulative
})

frequency_table
```

Out[13]:

	Absolute Frequency	Relative Frequency (%)	Cumulative Frequency
class			
First	216	24.24	216
Second	184	20.65	400
Third	491	55.11	891

Part II: Probability Analysis

Create the Contingency Table

In [14]: `pd.crosstab(df["sex"], df["survived"])`

Out[14]:

survived	0	1
----------	---	---

sex		
female	81	233
male	468	109

Store the Contingency Table

In [15]: `contingency = pd.crosstab(df["sex"], df["survived"])`
`contingency`

Out[15]:

survived	0	1
----------	---	---

sex		
female	81	233
male	468	109

Joint Probability

In [16]: `joint_probability = contingency / len(df)`
`joint_probability`

Out[16]:

survived	0	1
----------	---	---

sex		
female	0.090909	0.261504
male	0.525253	0.122334

Marginal Probability

```
In [17]: df["sex"].value_counts(normalize=True)
```

```
Out[17]: sex
male      0.647587
female    0.352413
Name: proportion, dtype: float64
```

```
In [18]: df["survived"].value_counts(normalize=True)
```

```
Out[18]: survived
0      0.616162
1      0.383838
Name: proportion, dtype: float64
```

Conditional Probability

```
In [19]: pd.crosstab(
    df["sex"],
    df["survived"],
    normalize="index"
)
```

```
Out[19]: survived      0      1
sex
female  0.257962  0.742038
male    0.811092  0.188908
```

Part III: Correlation Analysis

Select Required Columns

```
In [20]: df[["age", "fare"]].head()
```

```
Out[20]:   age  fare
0  22.0  7.2500
1  38.0 71.2833
2  26.0  7.9250
3  35.0 53.1000
4  35.0  8.0500
```

Check Missing Values

```
In [21]: df[["age", "fare"]].isnull().sum()
```

```
Out[21]: age      177
fare        0
dtype: int64
```

Remove Missing Values

```
In [22]: corr_data = df[["age", "fare"]].dropna()

corr_data.head()
```

```
Out[22]:
```

	age	fare
0	22.0	7.2500
1	38.0	71.2833
2	26.0	7.9250
3	35.0	53.1000
4	35.0	8.0500

Calculate Pearson Correlation

```
In [23]: corr_data.corr()
```

```
Out[23]:
```

	age	fare
age	1.000000	0.096067
fare	0.096067	1.000000

Store the Correlation Matrix

```
In [24]: correlation = corr_data.corr()

correlation
```

```
Out[24]:
```

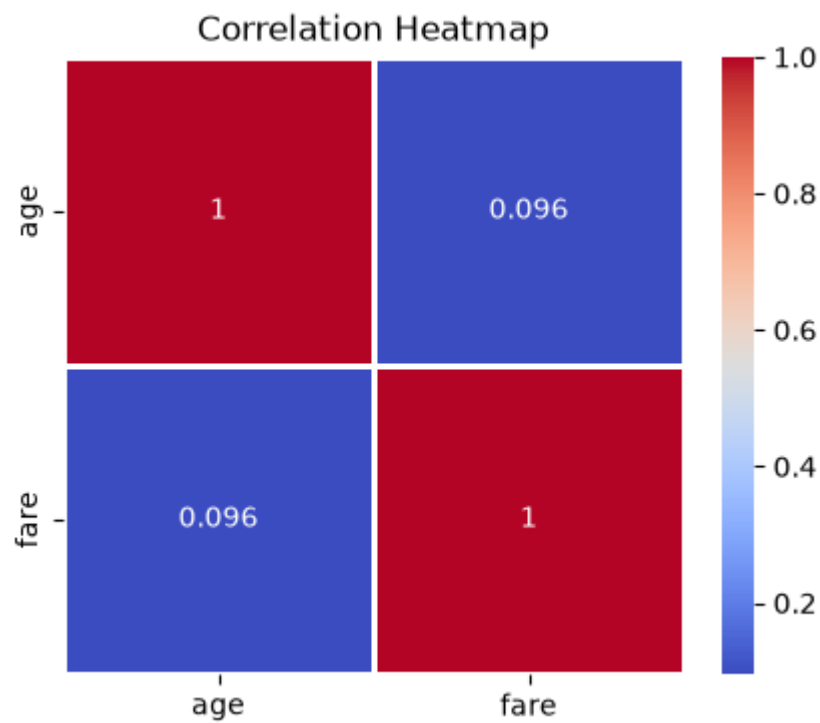
	age	fare
age	1.000000	0.096067
fare	0.096067	1.000000

Draw the Heatmap

```
In [25]: plt.figure(figsize=(5,4))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    linewidths=1
)
```

```
plt.title("Correlation Heatmap")
plt.show()
```



Scatter Plot

```
In [26]: plt.figure(figsize=(7,5))

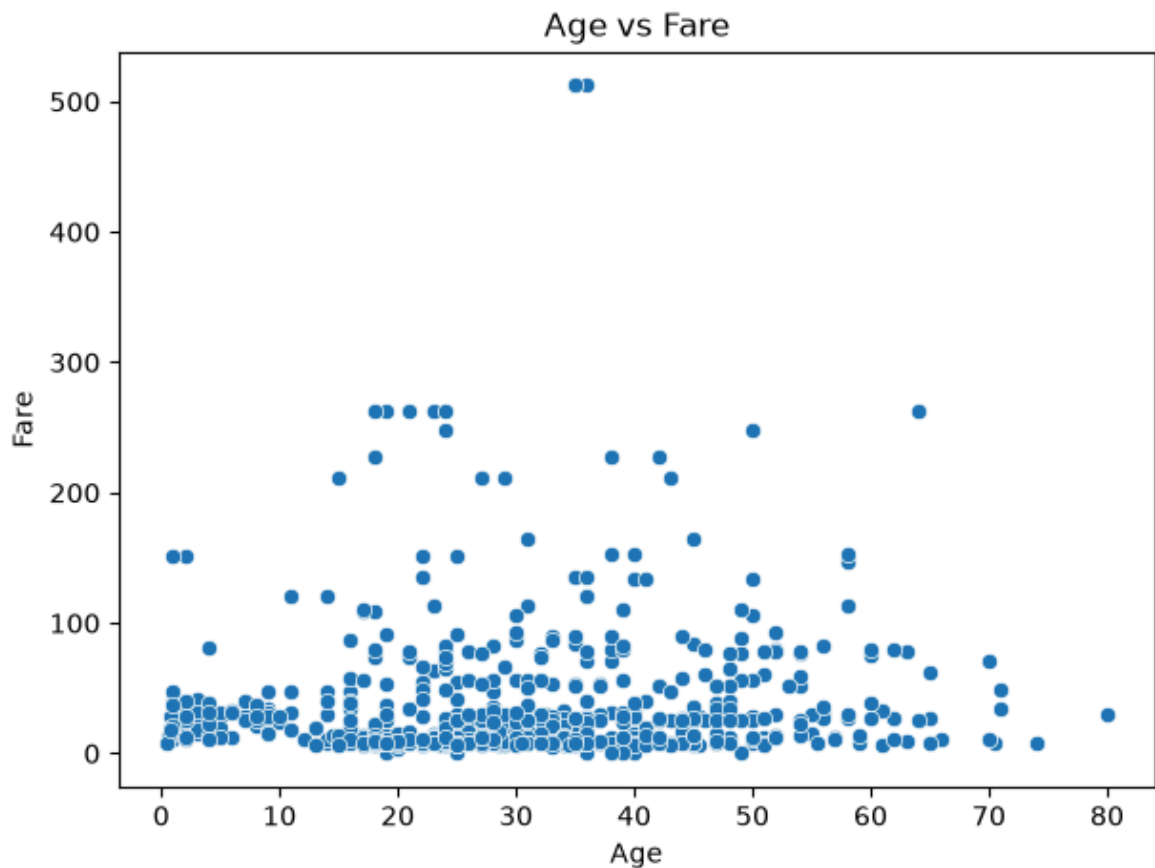
sns.scatterplot(
    data=corr_data,
    x="age",
    y="fare"
)

plt.title("Age vs Fare")

plt.xlabel("Age")

plt.ylabel("Fare")

plt.show()
```



Stacked Bar Chart

```
In [27]: survival_class = pd.crosstab(df["class"], df["survived"])

survival_class
```

```
Out[27]:
```

	survived	0	1
class			
First	80	136	
Second	97	87	
Third	372	119	

```
In [28]: survival_class.plot(
    kind="bar",
    stacked=True,
    figsize=(7,5)
)

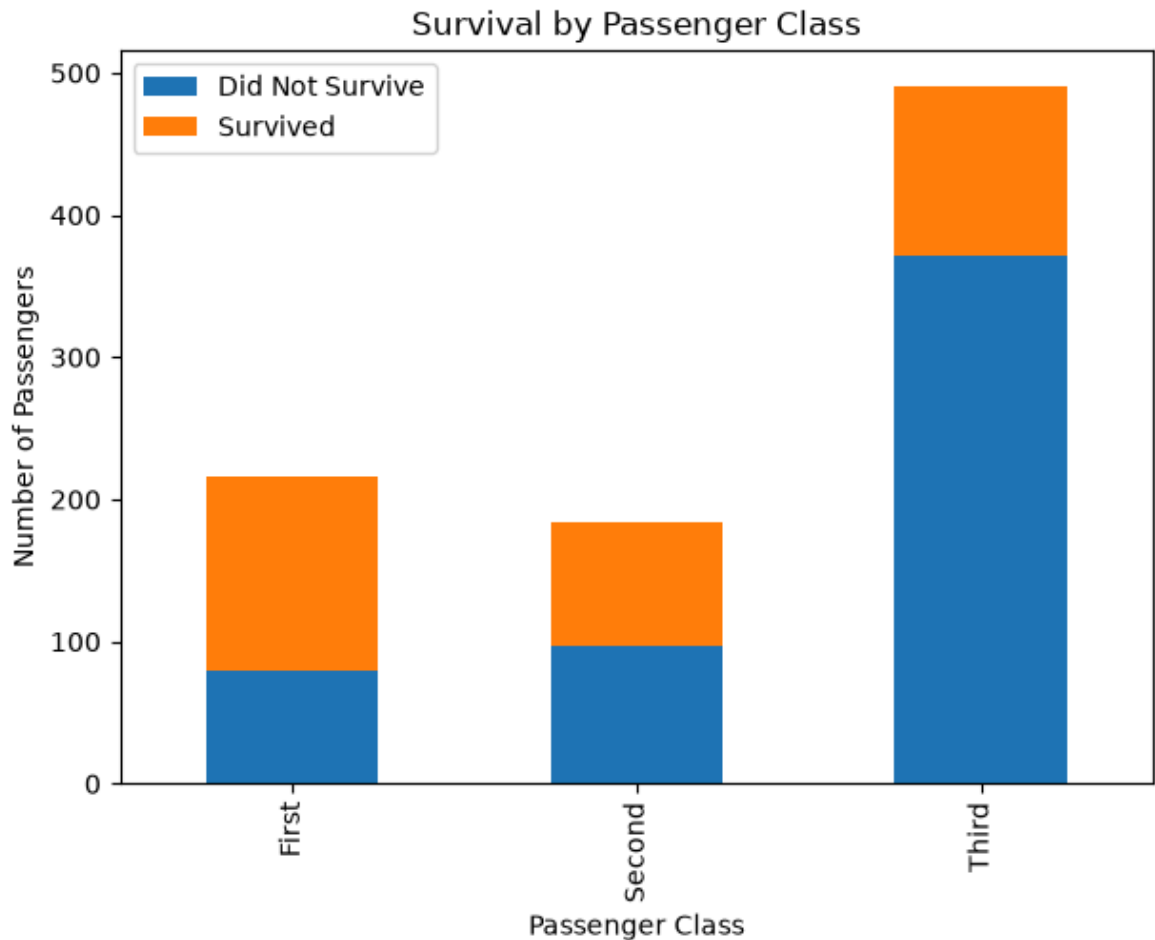
plt.title("Survival by Passenger Class")

plt.xlabel("Passenger Class")

plt.ylabel("Number of Passengers")

plt.legend(["Did Not Survive", "Survived"])

plt.show()
```



Assignment Questions

Frequency table for Embarked

```
In [30]: df["embarked"].value_counts()
```

```
Out[30]: embarked  
S      644  
C      168  
Q       77  
Name: count, dtype: int64
```

Probability that a passenger survived

```
In [31]: df["survived"].mean()
```

```
Out[31]: np.float64(0.3838383838383838)
```

Pearson Correlation

```
In [32]: correlation.loc["age", "fare"]
```

```
Out[32]: np.float64(0.09606669176903912)
```